IndexShare (paper: IndexCache)

Reuse one DSA indexer's top-k across layers

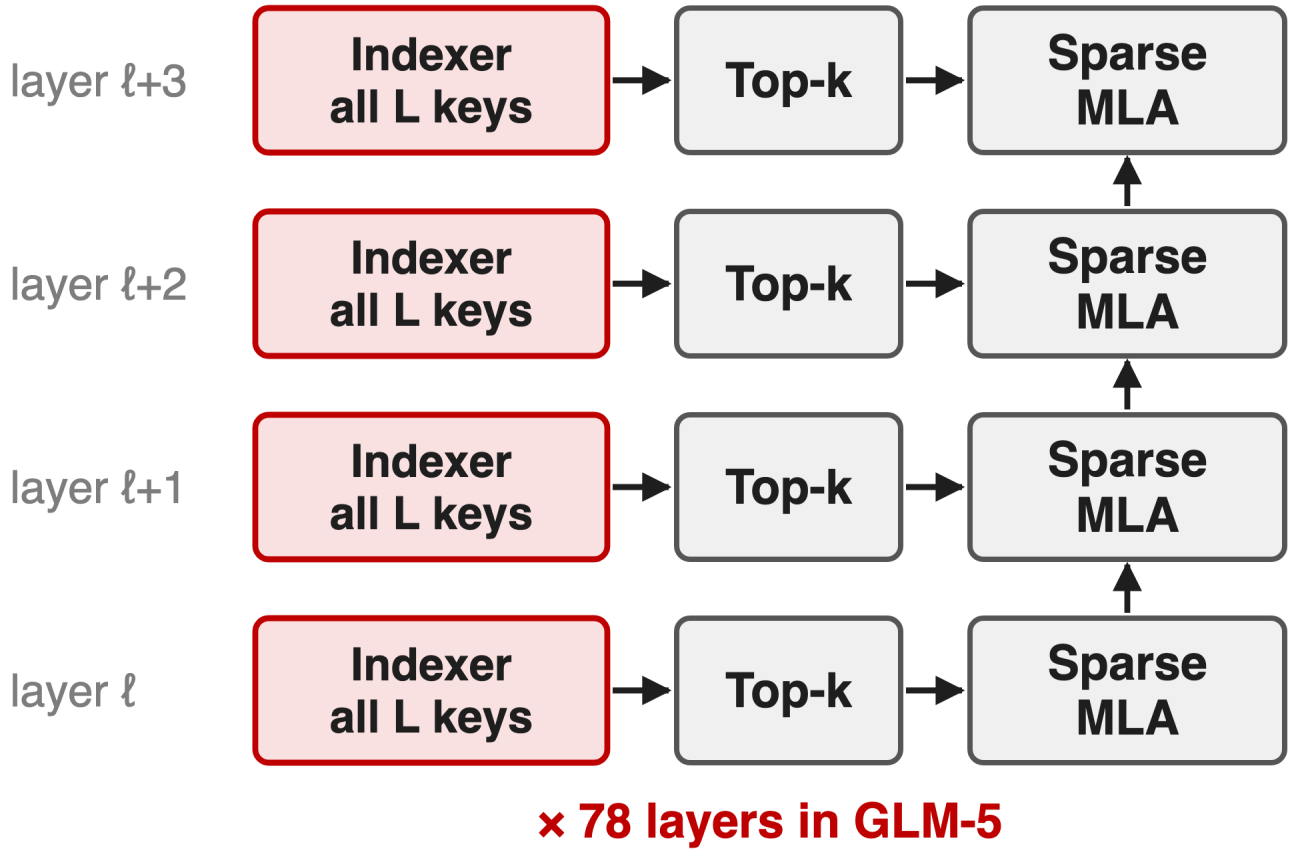
Tsinghua University & Z.ai · IndexCache paper; shipped in GLM-5.2

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Attention & Long Context · arxiv.org/abs/2603.12201

Every DSA layer runs its own indexer

- GLM-5: **744B**, **78** DSA layers, top-**2,048** (deck 02)
- GLM-5.2: context **200K** → **1M**
- Paper: **IndexCache** · GLM-5.2: **IndexShare**



At 200K, the indexer is 81% of prefill

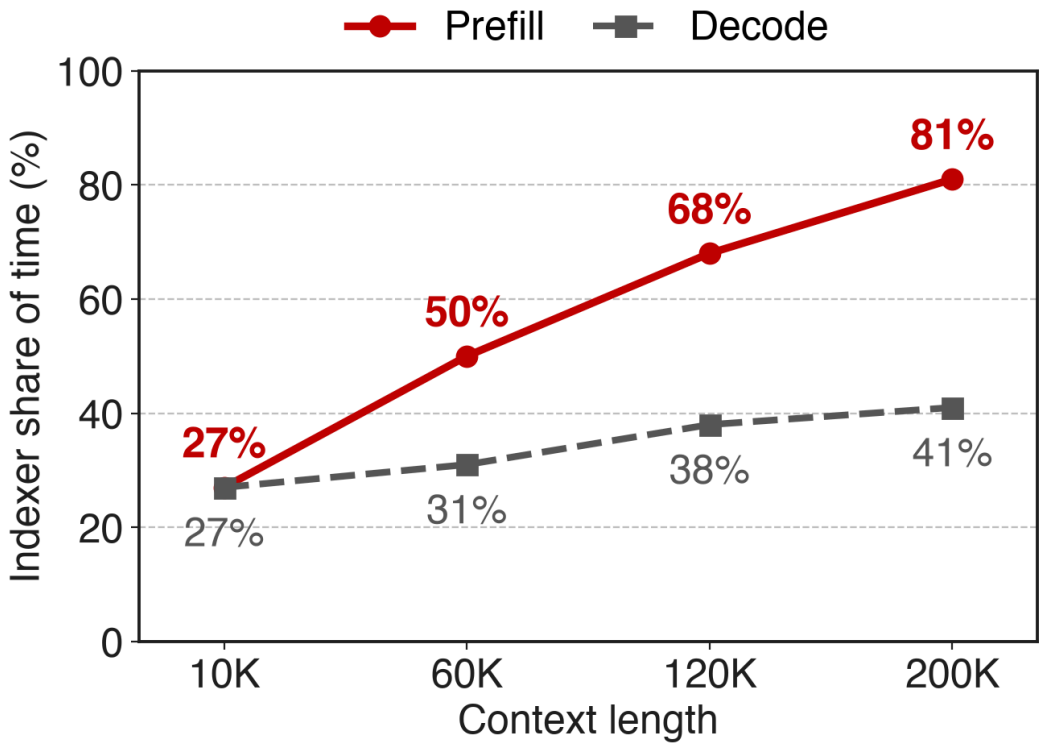
81%

of prefill time goes to the lightning indexers at 200K tokens (27% at 10K), 30B DSA model

IndexCache, arXiv 2603.12201, §1 figure; 30B GLM-4.7-Flash-based DSA model on H100 with SGLang

The indexer's share grows with context

- N layers × all L keys: **$O(N \cdot L^2)$**
- Core attention: only $O(N \cdot L \cdot k)$
- Decode: **41%** at 200K



Villain: each layer re-derives its top-k from scratch

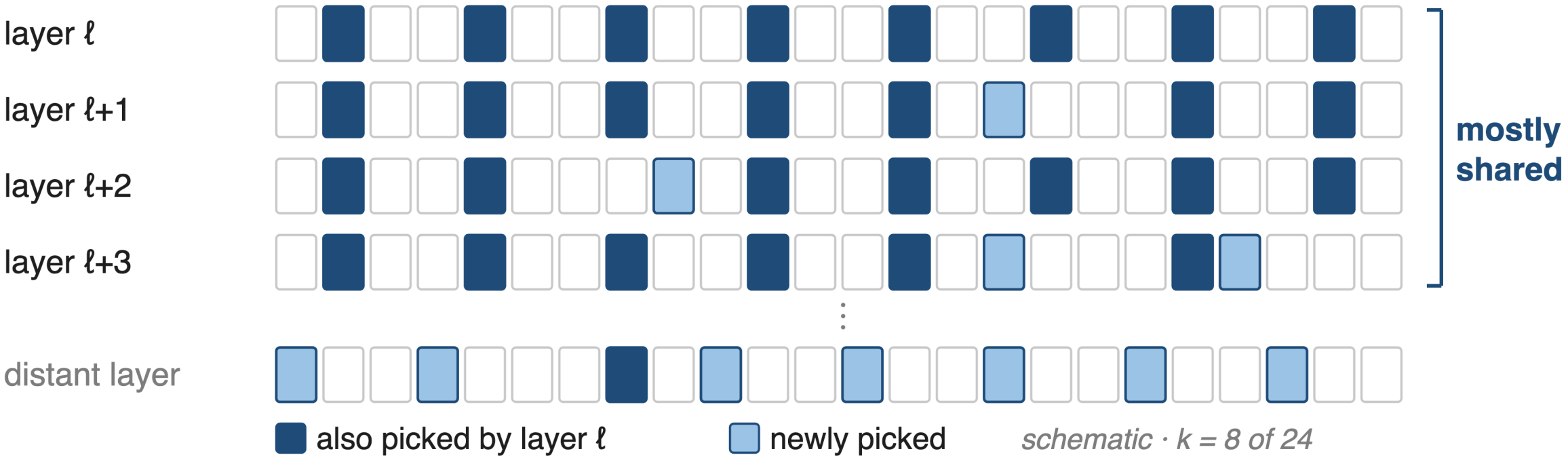
Prior reuse needs a full-attention oracle

- **Anchor-layer reuse** (TidalDecode, Kascade, HySparse)
- Copies top-k from **full-attention** layers
- DSA has **none left**

- **Skip indexers uniformly**
- Keep 1/4: long-context avg **–7.2**
- Pick by similarity: **no better**

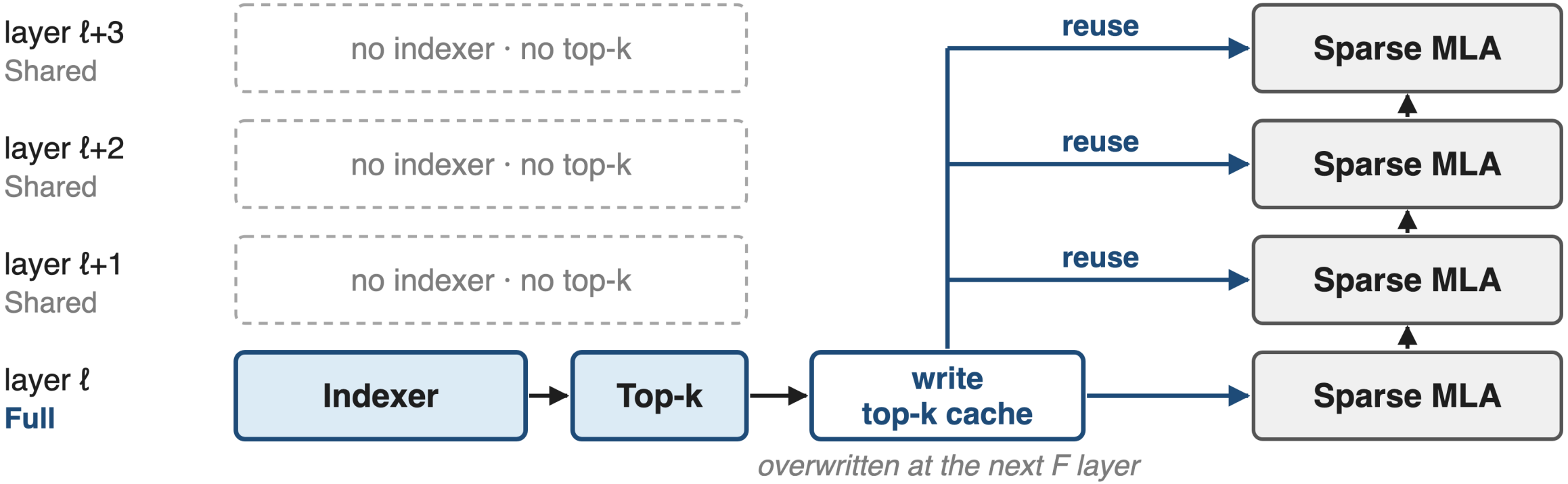
Needed: a cheap oracle DSA already has, and a rule for which layers keep it

Insight: adjacent layers share 70–100% of top-k



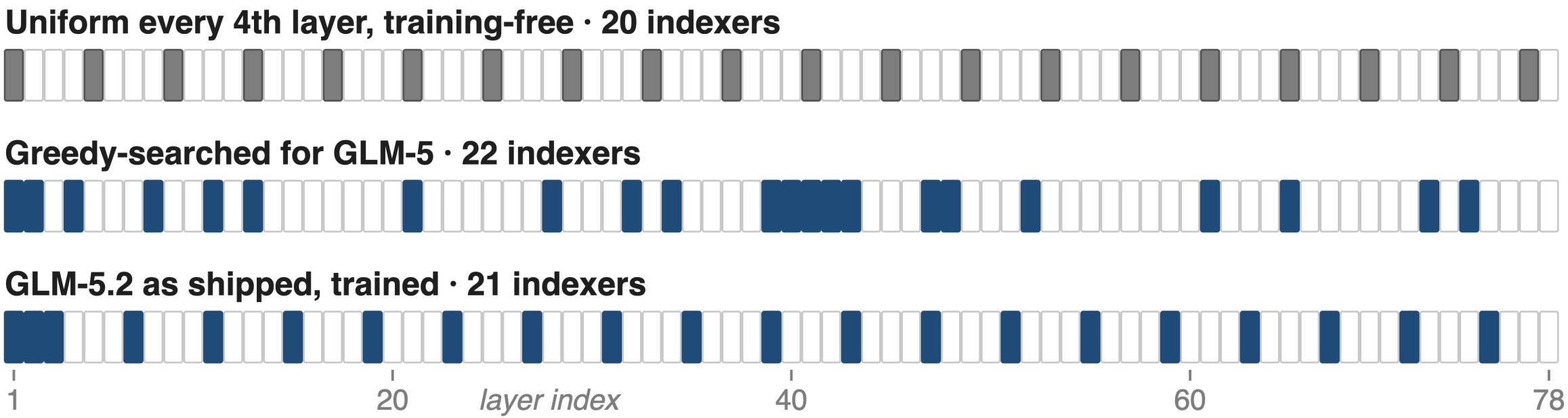
The indexer's own output is the oracle: compute it in a few layers, reuse it in the rest

❶ F layers index, S layers reuse the cache



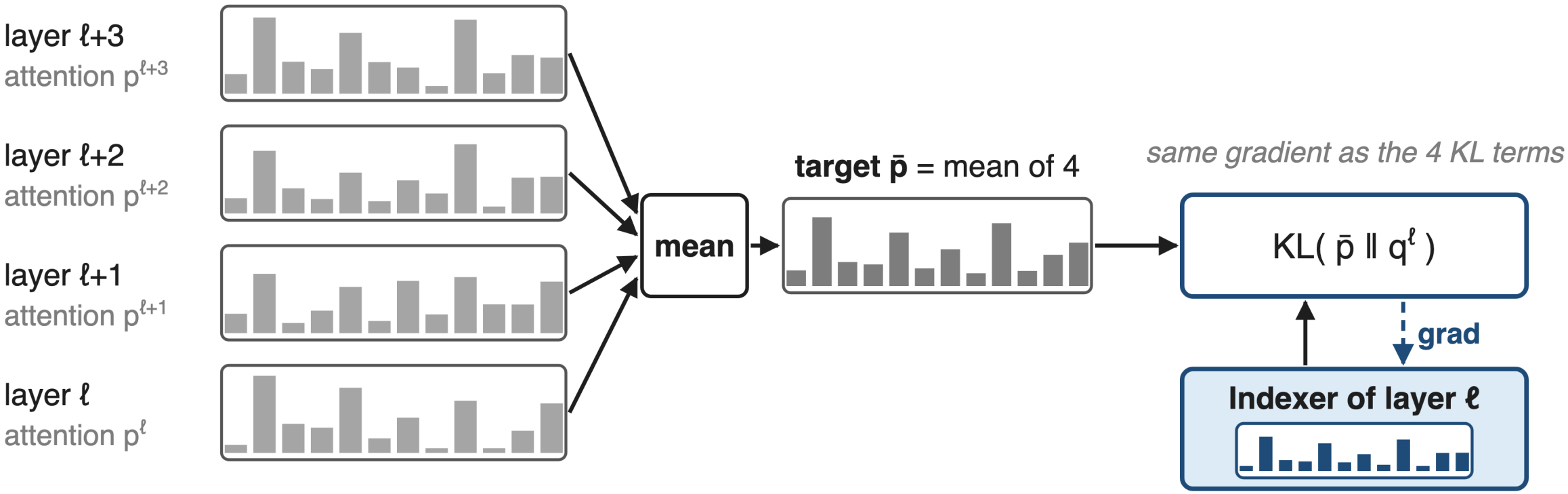
One branch per layer; the cache is overwritten at each F layer, so no extra memory

❷ Greedy search keeps the indexers that matter



Without retraining: flip one layer to S at a time, keeping the flip with the lowest calibration loss

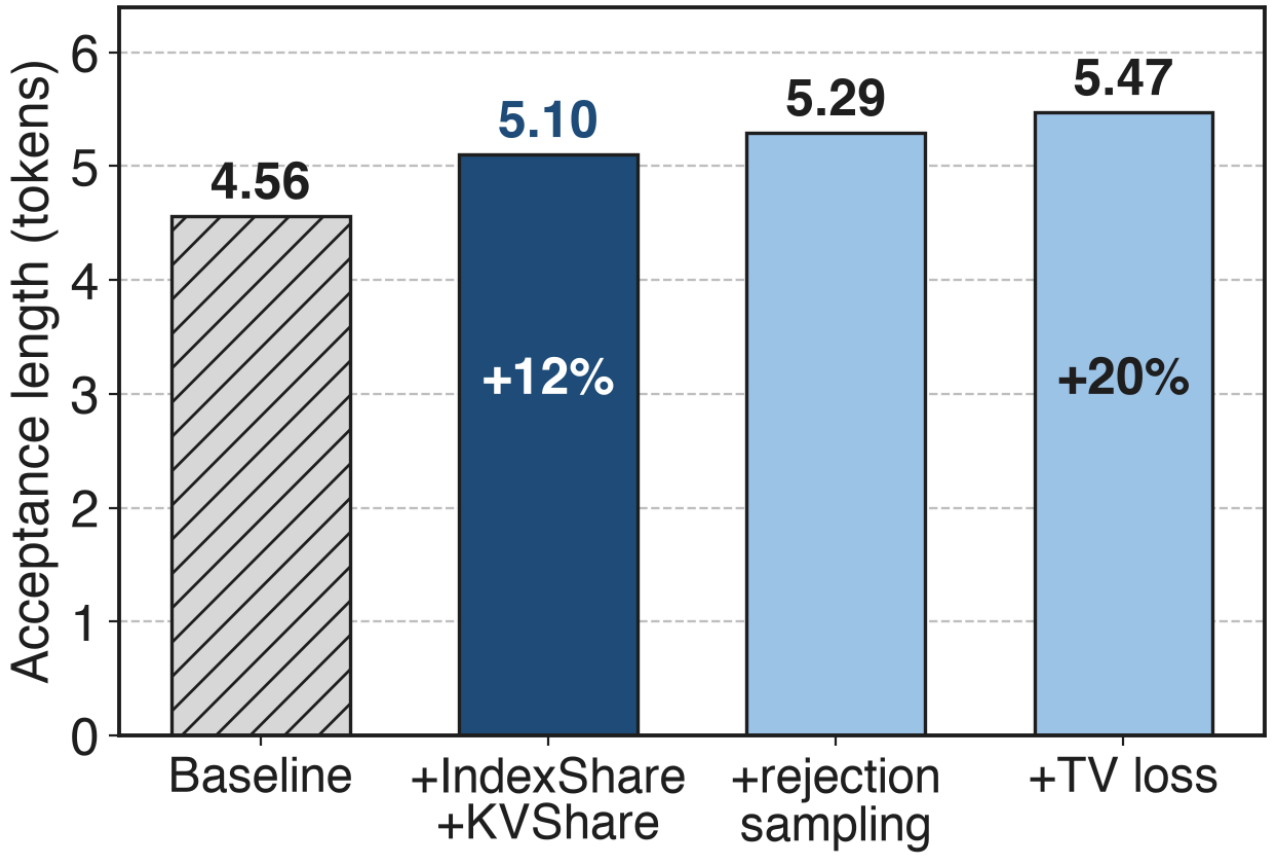
❸ Distil each indexer for every layer it serves



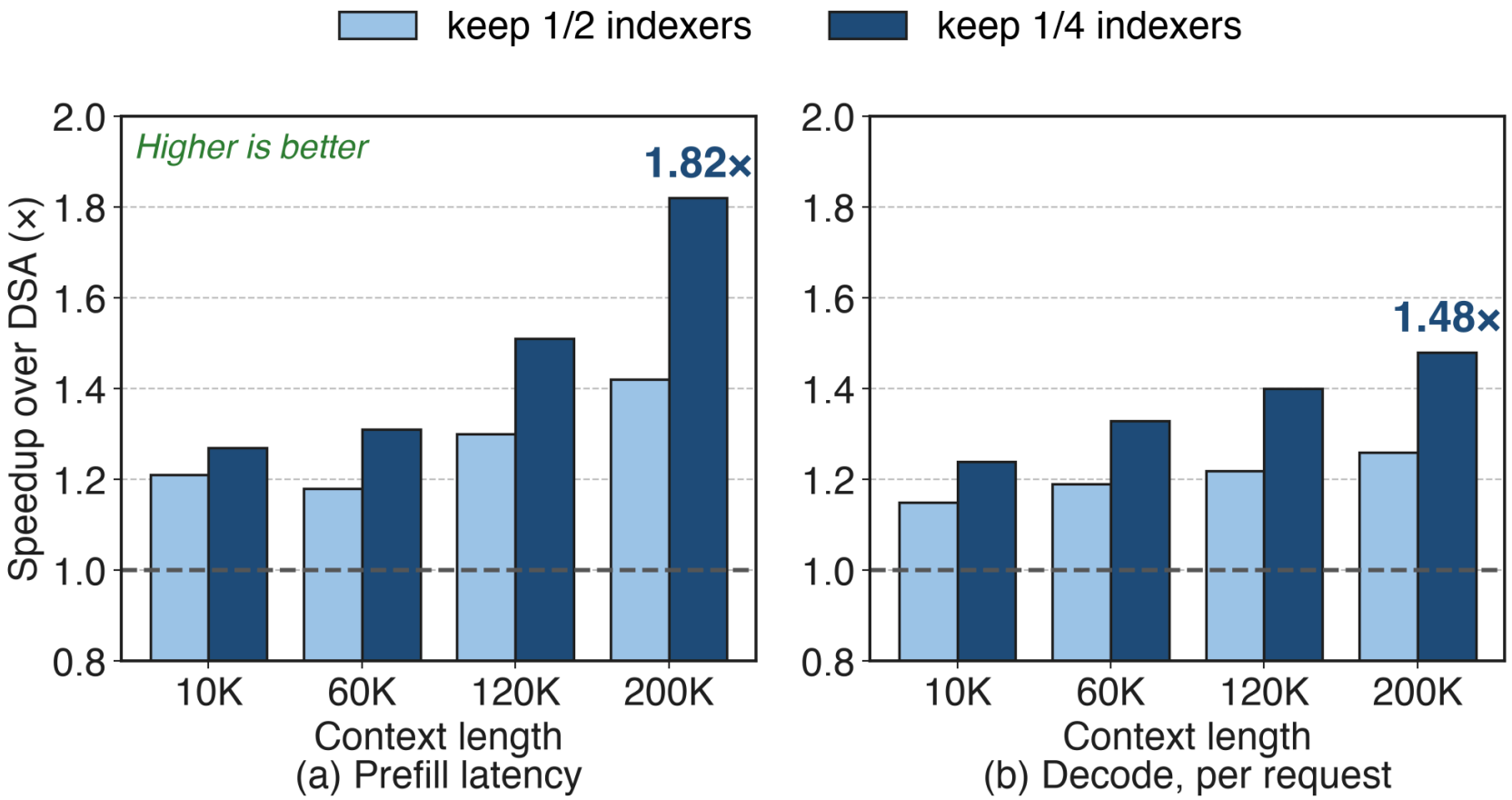
Trained this way, a plain every-4th pattern matches full DSA: GLM-5.2's choice

GLM-5.2's MTP drafts reuse indices and KV

- Indexer runs on the first MTP step only
- Later steps reuse its top-k and KV
- No mixed target/draft KV in later steps



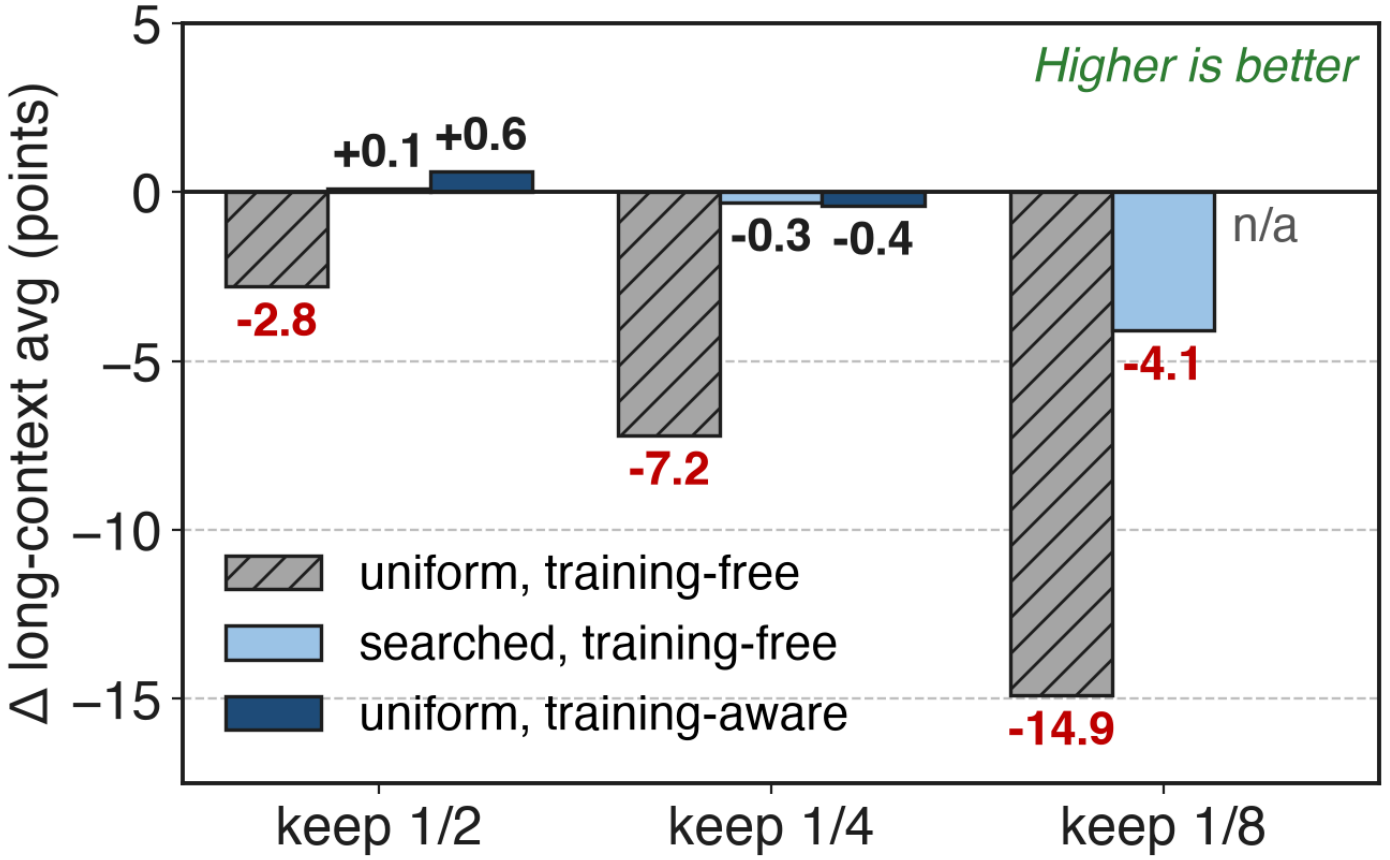
1/4 indexers: 1.82× prefill, 1.48× decode



30B DSA model, one H100 node, SGLang. At 200K: prefill 19.5 s → 10.7 s, per-request decode 58 → 86 tok/s

Searched or trained patterns lose ≤ 0.4 points

- Avg of 5 long-context tests, 30B
- Reasoning avg within **1** point
- GLM-5, searched 1/4: **78.0** vs **78.4**



GLM-5.2 ships it: 2.9× fewer FLOPs at 1M

2.9×

lower per-token FLOPs at 1M context, GLM-5.2 (21 indexers in 78 layers)
vs GLM-5.1 (an indexer in every layer)

Z.ai GLM-5.2 README and blog ("Lower FLOPs with IndexShare"); indexer layout from the GLM-5.2 HF config

The cost: search, retraining, a residual $O(L^2)$

- Search: **N(N–1)/2** forward passes (**3,003** for 78 layers)
- Keep 1/8: **–4.1** even when searched
- Kept indexers stay $O(L^2)$; KV cache barely shrinks
- Uniform patterns need retraining

Each remaining indexer still needs a fast kernel: deck 10

Summary

- DSA's indexers: **81%** of prefill time at 200K
- **Insight**: adjacent layers pick **70–100%** the same tokens
- ❶ F/S reuse ❷ greedy search ❸ multi-layer distillation
- **1.82×** prefill, **1.48×** decode at 1/4; GLM-5.2: **2.9×** fewer FLOPs at 1M

https://arxiv.org/abs/2603.12201